

Problem Session 4: Memory Class and API in C Programming

Kiatissak Markmeeshap	67070501005
Tithinan Sobking	67070501018
Worawut Sereethai	67070501040
Chanya Poolketkij	67070501058

Everything below was run on one machine, as the sheet requires: Ubuntu under WSL2 on Windows 11. The programs were compiled with exactly the two commands the sheet gives, `gcc -o exe src.c` and `gcc -no-pie -o exe src.c`, with no additional flags, so that nothing about the compilation can be blamed for the addresses that follow. Items 1 and 2 were built both ways and run three times per build; item 3 is run without `-no-pie`, as the sheet specifies.

1 Static Storage Class

1.1 The original program

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     auto int x = 3;
6
7     while (x > 0)
8     {
9         static int y = 5;
10        y++;
11
12        printf("The value of y is %d\n", y);
13        printf("The address of y is %p\n\n", (void *)&y);
14
15        x--;
16    }
17
18    return 0;
19 }
```

Listing 1: Source: src/1_static.c

```

--- 1_static.c, default PIE, run 1 ---
The value of y is 6
The address of y is 0x58cb201ad010

The value of y is 7
The address of y is 0x58cb201ad010

The value of y is 8
The address of y is 0x58cb201ad010

--- 1_static.c, default PIE, run 2 ---
The value of y is 6
The address of y is 0x5a3ca3532010

The value of y is 7
The address of y is 0x5a3ca3532010

The value of y is 8
The address of y is 0x5a3ca3532010

--- 1_static.c, default PIE, run 3 ---
The value of y is 6
The address of y is 0x5fc5b0b79010

The value of y is 7
The address of y is 0x5fc5b0b79010

The value of y is 8
The address of y is 0x5fc5b0b79010

```

Listing 2: All three runs of the original program

`static` gives `y` *static storage duration*: it is created once, before the program starts running, and lives in the initialised-data segment rather than on the stack. The initialiser `= 5` is therefore not an instruction executed each time control passes it — it is the value the variable is born with. Each iteration only runs `y++`, so the value goes 6, 7, 8, and the address is the same on all three lines because there is only ever one `y`.

(The `auto` on `x` is what every local already is, so it changes nothing; the keyword is a leftover from early C.)

1.2 With `static` removed

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     auto int x = 3;
6
7     while (x > 0)
8     {
9         int y = 5;
10        y++;
11
12        printf("The value of y is %d\n", y);
13        printf("The address of y is %p\n\n", (void *)&y);
14
15        x--;
16    }
17
18    return 0;
19 }

```

Listing 3: Source: `src/1_static_no_static.c`

```

--- 1_static_no_static.c, default PIE, run 1 ---
The value of y is 6
The address of y is 0x7fff9688e0f0

The value of y is 6
The address of y is 0x7fff9688e0f0

The value of y is 6
The address of y is 0x7fff9688e0f0

--- 1_static_no_static.c, default PIE, run 2 ---
The value of y is 6
The address of y is 0x7ffee90a8000

The value of y is 6
The address of y is 0x7ffee90a8000

The value of y is 6
The address of y is 0x7ffee90a8000

--- 1_static_no_static.c, default PIE, run 3 ---
The value of y is 6
The address of y is 0x7ffc8f16a3e0

The value of y is 6
The address of y is 0x7ffc8f16a3e0

The value of y is 6
The address of y is 0x7ffc8f16a3e0

```

Listing 4: All three runs after removing `static`

Without `static`, `y` is an ordinary automatic variable. It is created when the loop body is entered and destroyed when the body ends, so `= 5` really does run every iteration and the increment always starts over: the value is 6 three times instead of 6, 7, 8.

The address is now a stack address — far higher than before, in the `0x7ff...` range — and it repeats across the three iterations only because the same stack slot is handed back each time the body is re-entered. That is reuse, not persistence: the old value is gone even though the storage happens to be the same.

1.3 Compiled with `-no-pie`

```

--- 1_static.c, -no-pie, run 1 ---
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

--- 1_static.c, -no-pie, run 2 ---
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

--- 1_static.c, -no-pie, run 3 ---
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

```

Listing 5: The original program rebuilt with `-no-pie`

```

--- 1_static_no_static.c, -no-pie, run 1 ---
The value of y is 6
The address of y is 0x7fffd2a525e0

The value of y is 6
The address of y is 0x7fffd2a525e0

The value of y is 6
The address of y is 0x7fffd2a525e0

--- 1_static_no_static.c, -no-pie, run 2 ---
The value of y is 6
The address of y is 0x7ffe40c85ce0

The value of y is 6
The address of y is 0x7ffe40c85ce0

The value of y is 6
The address of y is 0x7ffe40c85ce0

--- 1_static_no_static.c, -no-pie, run 3 ---
The value of y is 6
The address of y is 0x7ffcca719fb0

The value of y is 6
The address of y is 0x7ffcca719fb0

The value of y is 6
The address of y is 0x7ffcca719fb0

```

Listing 6: The static-less program with `-no-pie`

Build	Run 1	Run 2	Run 3
static, default PIE	0x58cb201ad010	0x5a3ca3532010	0x5fc5b0b79010
static, -no-pie	0x404018	0x404018	0x404018
no static, default PIE	0x7fff9688e0f0	0x7ffee90a8000	0x7ffc8f16a3e0
no static, -no-pie	0x7fffd2a525e0	0x7ffe40c85ce0	0x7ffcca719fb0

Table 1: Address of `y` in each of the three runs

Yes, the results differ, and the table shows exactly where.

By default `gcc` produces a *position-independent executable*, which the loader is free to place at a randomly chosen base address; every address inside it shifts with that base, so the static `y` lands somewhere new on every run. `-no-pie` produces a fixed-position executable instead, linked to load at one address, so `y` is at `0x404018` in all three runs. The *values* 6, 7, 8 never change — storage duration is a language rule and has nothing to do with how the image is loaded.

The last row is the useful one: without `static`, the address still moves between runs even under `-no-pie`. That variable is on the stack, and the stack is randomised by address-space layout randomisation, which the kernel applies independently of whether the executable is position-independent. Turning off PIE fixes where the *program* is loaded; it does not turn ASLR off.

2 Extern Storage Class

2.1 The original program

```

1 #include <stdio.h>
2
3 int x = 20;
4
5 static void display(void)
6 {
7     extern int x;
8     printf("Display value of x: %d\n", x);
9     printf("Address of x (in display function): %p\n\n", (void *)&x);
10 }
11
12 int main(void)
13 {
14     extern int x;
15     printf("Print value of x: %d\n", x);
16     printf("Address of x (in main function): %p\n\n", (void *)&x);
17     display();
18     return 0;
19 }

```

Listing 7: Source: `src/2_extern.c`

```

--- 2_extern.c, default PIE, run 1 ---
Print value of x: 20
Address of x (in main function): 0x5e847872a010

Display value of x: 20
Address of x (in display function): 0x5e847872a010

--- 2_extern.c, default PIE, run 2 ---
Print value of x: 20
Address of x (in main function): 0x55bd30571010

Display value of x: 20
Address of x (in display function): 0x55bd30571010

--- 2_extern.c, default PIE, run 3 ---
Print value of x: 20
Address of x (in main function): 0x60475102d010

Display value of x: 20
Address of x (in display function): 0x60475102d010

```

Listing 8: All three runs of the original program

`extern int x;` *declares* `x` without defining it: it tells the compiler that a variable of that name and type exists somewhere and that this function should use it, rather than creating a new one. The definition is the file-scope `int x = 20;`, which is what actually reserves storage.

Both functions therefore name the same object, so both print 20 and both print the same address. Here the `extern` declarations are redundant — the file-scope definition is already visible to both functions — but they make the intent explicit, and in a real program they are how one translation unit reaches a variable defined in another.

2.2 With `extern` removed from `main()`

```

1  #include <stdio.h>
2
3  int x = 20;
4
5  static void display(void)
6  {
7      extern int x;
8      printf("Display value of x: %d\n", x);
9      printf("Address of x (in display function): %p\n\n", (void *)&x);
10 }
11
12 int main(void)
13 {
14     int x;
15     printf("Print value of x: %d\n", x);
16     printf("Address of x (in main function): %p\n\n", (void *)&x);
17     display();
18     return 0;
19 }

```

Listing 9: Source: `src/2_extern_no_extern.c`

```

--- 2_extern_no_extern.c, default PIE, run 1 ---
Print value of x: 0
Address of x (in main function): 0x7ffcd6c9e714

Display value of x: 20
Address of x (in display function): 0x56f9dc7b3010

--- 2_extern_no_extern.c, default PIE, run 2 ---
Print value of x: 0
Address of x (in main function): 0x7ffdc5b931d4

Display value of x: 20
Address of x (in display function): 0x586cd516f010

--- 2_extern_no_extern.c, default PIE, run 3 ---
Print value of x: 0
Address of x (in main function): 0x7ffc34bb6704

Display value of x: 20
Address of x (in display function): 0x5bd01b79b010

```

Listing 10: All three runs after removing `extern`

Deleting one keyword changes the meaning completely. `int x;` inside `main()` is no longer a declaration referring to the global — it is a *definition* of a new automatic variable that shadows it for the rest of the function. It is never assigned, so reading it is undefined behaviour; the compiler says so as well, warning ‘x’ is used uninitialized.

So `main()` prints whatever happened to be in that stack slot, and a stack address to go with it, while `display()` is untouched and still reports the global’s 20 at the global’s address. The two addresses printed in a single run are in completely different regions of memory, which is the clearest evidence that they are two different variables.

2.3 Compiled with `-no-pie`

```

--- 2_extern.c, -no-pie, run 1 ---
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

--- 2_extern.c, -no-pie, run 2 ---
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

--- 2_extern.c, -no-pie, run 3 ---
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

```

Listing 11: The original program rebuilt with `-no-pie`

```

--- 2_extern_no_extern.c, -no-pie, run 1 ---
Print value of x: 0
Address of x (in main function): 0x7fff9a126b64

Display value of x: 20
Address of x (in display function): 0x404020

--- 2_extern_no_extern.c, -no-pie, run 2 ---
Print value of x: 0
Address of x (in main function): 0x7ffe59e39194

Display value of x: 20
Address of x (in display function): 0x404020

--- 2_extern_no_extern.c, -no-pie, run 3 ---
Print value of x: 0
Address of x (in main function): 0x7ffebaabcea4

Display value of x: 20
Address of x (in display function): 0x404020

```

Listing 12: The extern-less program with `-no-pie`

Build	Run 1	Run 2	Run 3
global x, default PIE	0x5e847872a010	0x55bd30571010	0x60475102d010
global x, <code>-no-pie</code>	0x404018	0x404018	0x404018
local x in main, default PIE	0x7ffcd6c9e714	0x7ffdc5b931d4	0x7ffc34bb6704
local x in main, <code>-no-pie</code>	0x7fff9a126b64	0x7ffe59e39194	0x7ffebaabcea4

Table 2: Address of x in each of the three runs

The behaviour is the same as in item 1, for the same reason. The global x is part of the loaded image, so under default PIE its address follows the randomised load base and changes every run, while `-no-pie` pins it at 0x404018 in all three.

The local x in the bottom two rows keeps moving under both, because it lives on the stack and the stack is randomised regardless of PIE. The values printed are equally unrepeatable — each run finds different leftover bytes in that slot — which is what undefined behaviour looks like in practice.

3 Memory Allocation

3.1 The original program

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      int *a;
7      float *b;
8      int c[10];
9
10     printf("\nAddress of Pointer\n");
11     printf(">>>p\n", (void *)&a);
12     printf(">>>p\n", (void *)&b);
13
14     printf("\nEffective Address\n");
15     printf(">>>p\n", (void *)a);
16     printf(">>>p\n", (void *)b);
17

```

```

18     a = malloc(10 * sizeof(int));
19     printf("\nAfter malloc Pointer a\n");
20     printf(">>%p\n", (void *)a);
21     printf(">>%p\n", (void *)&a[0]);
22     printf(">>%p\n", (void *)&a[9]);
23
24     printf("\nArray c\n");
25     printf(">>%p\n", (void *)c);
26     printf(">>%p\n", (void *)&c[0]);
27     printf(">>%p\n", (void *)&c[9]);
28
29     a = realloc(a, 1000 * sizeof(int));
30     printf("\nAfter realloc Pointer a\n");
31     printf(">>%p\n", (void *)a);
32     printf(">>%p\n", (void *)&a[0]);
33     printf(">>%p\n", (void *)&a[9]);
34     printf(">>%p\n", (void *)&a[999]);
35
36     free(a);
37     return 0;
38 }

```

Listing 13: Source: src/3_memory.c

```

--- 3_memory.c, default PIE, run 1 ---

Address of Pointer
>>0x7ffed482f730
>>0x7ffed482f738

Effective Address
>>0x1800000017
>>(nil)

After malloc Pointer a
>>0x558fb7513020
>>0x558fb7513020
>>0x558fb7513044

Array c
>>0x7ffed482f740
>>0x7ffed482f740
>>0x7ffed482f764

After realloc Pointer a
>>0x558fb7513020
>>0x558fb7513020
>>0x558fb7513044
>>0x558fb7513fbc

```

Listing 14: The program as given

Each section of the output means something different.

Address of Pointer is `&a` and `&b` — where the two pointer *variables* themselves sit. They are locals, so these are stack addresses, eight bytes apart because a pointer is eight bytes on this machine.

Effective Address is `a` and `b` — the values stored in those pointers, read before anything has been assigned to them. That is undefined behaviour: the values are whatever the stack happened to hold. In this run one printed `(nil)` and the other a nonsense value, which is exactly the point — `(nil)` is not a guarantee, and an uninitialised pointer is not a null pointer.

After malloc shows `a` pointing into the heap, a region far from the stack. `a` and `&a[0]` print the same thing, because that is what indexing means, and `&a[9]` is 36 bytes further along: nine ints of four bytes each.

Array c is a stack array, so it sits next to the other locals rather than in the heap. It needs no allocation and cannot be freed; its lifetime ends with the function.

`realloc` here returned the same address it was given. The block was the most recent allocation and nothing lay above it, so the allocator could simply extend it in place; `&a[999]` is then 3996 bytes past the start.

3.2 With the allocation of `b` enabled

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     int *a;
7     float *b;
8     int c[10];
9
10    printf("\nAddress of Pointer\n");
11    printf(">>>p\n", (void *)&a);
12    printf(">>>p\n", (void *)&b);
13
14    printf("\nEffective Address\n");
15    printf(">>>p\n", (void *)&a);
16    printf(">>>p\n", (void *)&b);
17
18    a = malloc(10 * sizeof(int));
19    printf("\nAfter malloc Pointer a\n");
20    printf(">>>p\n", (void *)&a);
21    printf(">>>p\n", (void *)&a[0]);
22    printf(">>>p\n", (void *)&a[9]);
23
24    printf("\nArray c\n");
25    printf(">>>p\n", (void *)&c);
26    printf(">>>p\n", (void *)&c[0]);
27    printf(">>>p\n", (void *)&c[9]);
28
29    b = malloc(10 * sizeof(float));
30    printf("\nAfter malloc Pointer b\n");
31    printf(">>>p\n", (void *)&b);
32    printf(">>>p\n", (void *)&b[0]);
33    printf(">>>p\n", (void *)&b[9]);
34
35    a = realloc(a, 1000 * sizeof(int));
36    printf("\nAfter realloc Pointer a\n");
37    printf(">>>p\n", (void *)&a);
38    printf(">>>p\n", (void *)&a[0]);
39    printf(">>>p\n", (void *)&a[9]);
40    printf(">>>p\n", (void *)&a[999]);
41
42    free(a);
43    free(b);
44    return 0;
45 }
```

Listing 15: Source: `src/3_memory_with_b.c`

```
--- 3_memory_with_b.c, default PIE, run 1 ---
```

```
Address of Pointer
```

```
>>0x7fffe767fea0
```

```
>>0x7fffe767fea8
```

```
Effective Address
```

```
>>0x1800000017
```

```
>>(nil)
```

```
After malloc Pointer a
```

```
>>0x5bf54a021020
```

```
>>0x5bf54a021020
```

```
>>0x5bf54a021044
```

```
Array c
```

```
>>0x7fffe767feb0
```

```
>>0x7fffe767feb0
```

```
>>0x7fffe767fed4
```

```
After malloc Pointer b
```

```
>>0x5bf54a021050
```

```
>>0x5bf54a021050
```

```
>>0x5bf54a021074
```

```
After realloc Pointer a
```

```
>>0x5bf54a021080
```

```
>>0x5bf54a021080
```

```
>>0x5bf54a0210a4
```

```
>>0x5bf54a02201c
```

Listing 16: The same program with **b** allocated

Build	After malloc	After realloc
without b	0x558fb7513020	0x558fb7513020
with b	0x5bf54a021020	0x5bf54a021080

Table 3: Where **a** lives before and after `realloc()`

Yes — with **b** allocated, **a** moves.

The first `malloc` still hands back the same kind of address, and **b**’s block is placed immediately after **a**’s. That is what changes the outcome: **a** is no longer the last thing in the heap, so growing it from 40 bytes to 4000 in place would run straight into **b**. The allocator has to find room elsewhere, copy the contents across, release the old block, and return a different pointer.

This is why `realloc` must be used as `a = realloc(a, ...)` and why any other pointer into the old block is invalid afterwards. Whether a block moves is not a property of the program but of what else has been allocated around it — the same call moved nothing in the previous run and moves the block here, with no change to the `realloc` line itself.